

Projekt: Auswirkungen von Wetterereignissen auf die urbane Mobilität in New York City

Team: Aron Milojevic, Mateusz Pacyga und Ajay Pal

Big Data Engineering Pipeline mit Apache Kafka, Apache Spark und MongoDB.

GitHub: https://github.com/milojevicaa/BD_NYC_Taxi_Weather

1. Einleitung & Story

New York City lebt von Bewegung. Millionen von Menschen bewegen sich täglich durch die Stadt, und das Yellow-Cab-Taxi ist seit Jahrzehnten ein Sinnbild dieser urbanen Mobilität. Doch wie reagiert die Taxinachfrage, wenn das Wetter umschlägt? Fahren die New Yorker bei Regen und Schnee häufiger Taxi, weil sie den Fußweg oder das Warten an der U-Bahn vermeiden wollen? Oder bleiben sie einfach zu Hause?

In diesem Projekt verbinden wir **drei unabhängige Datenquellen** zu einer durchgängigen Data-Engineering-Pipeline und beantworten die Frage:

Wie beeinflussen Wetterbedingungen (Temperatur, Regen, Schnee) die stündliche Taxinachfrage in New York City im Januar 2023?

Die Analyse selbst ist bewusst einfach gehalten – im Mittelpunkt steht die **technische Infrastruktur** (Kafka als zentraler Daten-Broker, Spark als Verarbeitungsschicht, MongoDB und Flat Files als persistente Speicher).

2. Big-Data-Kriterien (Theoretische Betrachtung)

Wir ordnen unser Vorhaben anhand der **5 Vs** und der **4 Ebenen der Datenverarbeitung** ein.

Die 5 Vs unseres Projekts

- **Volume:** Der NYC-TLC-Datensatz für Januar 2023 enthält rund **3 Millionen** einzelne Taxifahrten.
- **Velocity:** Taxidaten entstehen in der Realität kontinuierlich. Wir bilden diesen Echtzeit-Charakter ab, indem wir die Fahrten als **Event-Stream über Apache Kafka** an einen Broker senden.
- **Variety:** Wir kombinieren drei Akquisitionsarten – eine **Datei** (Parquet/CSV), eine **REST-API** (Open-Meteo) und **Web Scraping** (Wikipedia) – sowie strukturierte und semistrukturierte Formate.

- **Veracity:** Die Rohdaten enthalten Ausreißer (Fahrten ohne Distanz, negative Beträge, ungültige Zeitstempel). Diese werden in einem Cleaning-Schritt entfernt.
- **Value:** Die Verknüpfung von Nachfrage und Wetter ermöglicht eine bedarfsgerechte Flottensteuerung.

Die 4 Ebenen der Datenverarbeitung

1. **Data Source:** Datei (Parquet/CSV), REST-API (Open-Meteo) und Web Scraping (Wikipedia).
2. **Data Ingestion / Broker:** Apache Kafka mit den Topics `nyc_taxi_trips` und `nyc_weather`.
3. **Data Processing:** Apache Spark liest aus Kafka, parst die JSON-Events, aggregiert und joint.
4. **Data Storage & Output:** Persistenz in MongoDB (NoSQL) und Flat Files (Parquet/CSV), gefolgt von der Visualisierung in diesem Notebook.

3. Infrastruktur & Architektur

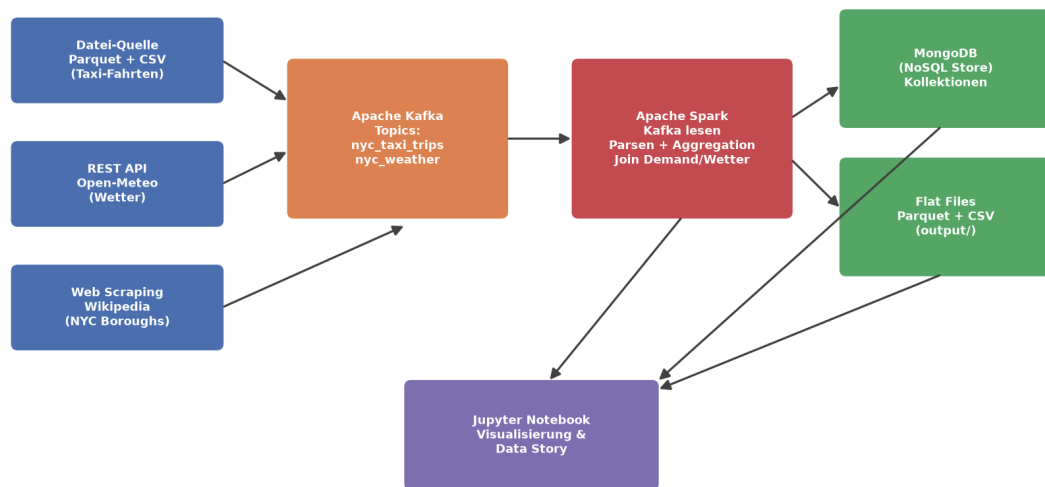
Die gesamte Infrastruktur ist über **Docker** gekapselt (`docker-compose.yml`), damit alle Teammitglieder eine identische Umgebung nutzen.

Dienst	Rolle	Port
Apache Kafka	Zentraler Daten-Broker (KRaft-Modus)	<code>localhost:29092</code>
Kafka-UI	Web-Oberfläche zur Inspektion der Topics	<code>localhost:8080</code>
MongoDB	NoSQL-Speicher für die Ergebnisse	<code>localhost:27017</code>
JupyterLab	Analyse, Dokumentation, Visualisierung	<code>localhost:8888</code>

Datenfluss-Diagramm

Das folgende Diagramm listet alle Datenquellen, die Transformationsschritte und die Ergebnisse auf.

NYC Taxi & Wetter - Big Data Datenfluss



Ablauf: Drei Datenquellen → **Kafka** (Topics) → **Spark** (Parsing, Aggregation, Join) → **MongoDB & Flat Files** → **Visualisierung** im Notebook.

4. Setup der Pipeline

Die wiederverwendbare Logik liegt im Paket `pipeline/` (Datenquellen, Kafka-Producer, Spark-Verarbeitung, Speicherung). Wir laden die Module und konfigurieren die Umgebung für Spark (JAVA_HOME und HADOOP_HOME mit `winutils` für die Ausführung unter Windows).

```
In [1]: import os
import sys

sys.path.insert(0, os.path.abspath("."))

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from pipeline import config, sources, kafka_pipeline, spark_pipeline, storage

config.configure_spark_environment()
sns.set_theme(style="whitegrid")

print("Kafka Broker :", config.KAFKA_BOOTSTRAP)
print("MongoDB URI  :", config.MONGO_URI)
print("Topics       :", config.TOPIC_TAXI, "|", config.TOPIC_WEATHER)
```

```
Kafka Broker : localhost:29092
MongoDB URI  : mongodb://localhost:27017/
Topics       : nyc_taxi_trips | nyc_weather
```

5. Datenquellen

Wir erfüllen die Anforderung von **drei unterschiedlichen Datenquellen mit unterschiedlicher Akquisitionsart**.

5.1 Quelle 1 – Datei (Parquet & CSV)

Die Kerndaten sind die **Yellow-Taxi-Fahrten** der NYC TLC (Parquet) sowie ein lokaler **Wetter-Snapshot** (CSV). Beim Laden bereinigen wir die Fahrten direkt: nur Januar 2023, nur Fahrten mit positiver Distanz, positivem Betrag und mindestens einem Fahrgast.

```
In [2]: taxi_df = sources.load_taxi_trips()
        weather_file_df = sources.load_weather_file()

        print(f"Bereinigte Taxi-Fahrten: {len(taxi_df):,}")
        print(f"Wetter-Snapshot (CSV) : {len(weather_file_df):,} Stundenwerte")
        taxi_df.head()
```

Bereinigte Taxi-Fahrten: 2,884,418
Wetter-Snapshot (CSV) : 744 Stundenwerte

```
Out[2]:
```

	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	fare_amount
0	2023-01-01 00:32:10	2023-01-01 00:40:36	1.0	0.97	
1	2023-01-01 00:55:08	2023-01-01 01:01:27	1.0	1.10	
2	2023-01-01 00:25:04	2023-01-01 00:37:49	1.0	2.51	
3	2023-01-01 00:10:29	2023-01-01 00:21:19	1.0	1.43	
4	2023-01-01 00:50:34	2023-01-01 01:02:52	1.0	1.84	

```
In [3]: weather_file_df[["datetime", "temp", "precip", "snow", "conditions"]].head()
```

```
Out[3]:
```

	datetime	temp	precip	snow	conditions
0	2023-01-01 00:00:00	12.2	0.185	0.0	Rain, Overcast
1	2023-01-01 01:00:00	11.7	0.028	0.0	Rain, Overcast
2	2023-01-01 02:00:00	11.7	0.094	0.0	Rain, Partially cloudy
3	2023-01-01 03:00:00	11.1	0.000	0.0	Partially cloudy
4	2023-01-01 04:00:00	11.4	0.000	0.0	Partially cloudy

5.2 Quelle 2 – REST API (Open-Meteo)

Über die **Open-Meteo Archive REST-API** beziehen wir die historischen, stündlichen Wetterdaten für NYC im Januar 2023 programmatisch (Temperatur, Niederschlag, Regen, Schneefall, Wind). Diese API-Quelle dient als die **streaming-fähige** Wetterquelle, die wir anschließend über Kafka verteilen.

```
In [4]: weather_api_df = sources.fetch_weather_api()

        print(f"Wetter-Datensätze von der API: {len(weather_api_df):,}")
        weather_api_df.head()
```

Out[4]:

	datetime	temp_c	precip_mm	rain_mm	snow_cm	wind_kmh	source
0	2023-01-01 00:00:00	10.9	1.0	1.0	0.0	NaN	open-meteo-api
1	2023-01-01 01:00:00	10.6	1.0	1.0	0.0	NaN	open-meteo-api
2	2023-01-01 02:00:00	10.6	0.1	0.1	0.0	NaN	open-meteo-api
3	2023-01-01 03:00:00	10.5	0.0	0.0	0.0	NaN	open-meteo-api
4	2023-01-01 04:00:00	9.8	0.0	0.0	0.0	NaN	open-meteo-api

5.3 Quelle 3 – Web Scraping (Wikipedia)

Mit `requests` und `BeautifulSoup` extrahieren wir die Referenztablette der **fünf Stadtbezirke (Boroughs)** von New York City aus Wikipedia. Diese Dimension (Einwohnerzahl, Fläche, Dichte) liefert den geografischen Kontext für die Größenordnung der städtischen Mobilität.

In [5]: `boroughs_df = sources.scrape_nyc_boroughs()`
`boroughs_df`

Out[5]:

	borough	county	population_2020	land_area_sqmi	land_area_sqkm	density_per
0	The Bronx	Bronx	1472654.0	42.2	109.2	34
1	Brooklyn	Kings	2736074.0	69.4	179.7	39
2	Manhattan	New York	1694251.0	22.7	58.7	74
3	Queens	Queens	2405464.0	108.7	281.6	22
4	Staten Island	Richmond	495747.0	57.5	149.0	8

6. Kafka-Ingestion

Nun machen wir die Daten über den **Kafka-Broker** verfügbar. Wir erzeugen die Topics neu (damit das Notebook wiederholt ausführbar bleibt) und schreiben mit den **Kafka-Producern** die Wetter-Events sowie eine repräsentative Stichprobe der Taxi-Fahrten als JSON in die Topics.

In [6]: `kafka_pipeline.recreate_topic(config.TOPIC_WEATHER)`
`weather_sent = kafka_pipeline.produce_weather(weather_api_df)`
`print(f"{weather_sent} Wetter-Events -> Topic '{config.TOPIC_WEATHER}')`

744 Wetter-Events -> Topic 'nyc_weather'

```
In [7]: taxi_sample = taxi_df.sample(n=min(config.TRIP_SAMPLE_SIZE, len(taxi_df)), random_state=12345)

kafka_pipeline.recreate_topic(config.TOPIC_TAXI)
trips_sent = kafka_pipeline.produce_taxi_trips(taxi_sample)
print(f"{trips_sent:,} Fahrt-Events -> Topic '{config.TOPIC_TAXI}'")

120,000 Fahrt-Events -> Topic 'nyc_taxi_trips'
```

7. Verarbeitung mit Apache Spark

Apache Spark liest die Events **aus den Kafka-Topics** (Batch-Read, `startingOffsets=earliest`), parst die JSON-Nachrichten in ein typisiertes Schema und führt die eigentliche Verarbeitung durch:

- **Map / Parsing:** Jedes Kafka-Event wird in strukturierte Spalten überführt, der Zeitstempel wird auf die volle Stunde gerundet.
- **Reduce / Aggregation:** Wir aggregieren die Nachfrage pro Stunde und pro Zahlungsart.
- **Join:** Die stündliche Nachfrage wird mit den Wetterdaten zusammengeführt und nach Wetterkategorie verdichtet.

```
In [8]: spark = spark_pipeline.build_spark()
print("Spark Version:", spark.version)

trips = spark_pipeline.read_trips_from_kafka(spark)
weather = spark_pipeline.read_weather_from_kafka(spark)

print("Gelesene Fahrt-Events :", trips.count())
print("Gelesene Wetter-Events :", weather.count())
```

```
Spark Version: 3.5.1
Gelesene Fahrt-Events : 120000
Gelesene Wetter-Events : 744
```

```
In [9]: hourly_demand = spark_pipeline.aggregate_hourly_demand(trips)
payment_mix = spark_pipeline.aggregate_payment_mix(trips)
joined = spark_pipeline.join_demand_with_weather(hourly_demand, weather)
by_weather = spark_pipeline.demand_by_weather_category(joined)

by_weather.show()
```

```
+-----+-----+-----+-----+
|weather_category|total_trips|avg_trips_per_hour|avg_temp_c|
+-----+-----+-----+-----+
|Snow|1227|175.3|2.0|
|Rain|21330|166.6|6.3|
|Dry|97443|160.0|4.4|
+-----+-----+-----+-----+
```

```
In [10]: hourly_pd = joined.toPandas()
payment_pd = payment_mix.toPandas()
weather_pd = by_weather.toPandas()

hourly_pd["date_hour"] = pd.to_datetime(hourly_pd["date_hour"])
hourly_pd["hour"] = hourly_pd["date_hour"].dt.hour
```

```
print("Spark-Ergebnis (stündlich, mit Wetter gejoint):", hourly_pd.shape)
hourly_pd.head()
```

Spark-Ergebnis (stündlich, mit Wetter gejoint): (744, 11)

```
Out[10]:
```

	date_hour	trip_count	avg_distance	avg_total_amount	revenue	temp_c	precip_mm
0	2023-01-01 00:00:00	191	3.545	28.23	5392.50	10.9	1.0
1	2023-01-01 01:00:00	201	3.170	26.00	5225.73	10.6	1.0
2	2023-01-01 02:00:00	165	3.171	25.45	4199.00	10.6	0.1
3	2023-01-01 03:00:00	123	3.713	26.51	3261.01	10.5	0.0
4	2023-01-01 04:00:00	86	3.913	28.31	2434.68	9.8	0.0

8. Speicherung der Ergebnisse (ETL-Output)

Die transformierten Ergebnisse werden für die spätere Nutzung persistiert – sowohl in **MongoDB** (NoSQL-Datenbank) als auch als **Flat Files** (Parquet und CSV im Ordner `output/`).

```
In [11]: mongo_hourly = storage.save_to_mongo(hourly_pd, "demand_weather_hourly")
mongo_payment = storage.save_to_mongo(payment_pd, "payment_mix")
mongo_weather = storage.save_to_mongo(weather_pd, "demand_by_weather")
storage.save_to_mongo(boroughs_df, "nyc_boroughs")

print(f"MongoDB 'demand_weather_hourly': {mongo_hourly} Dokumente")
print(f"MongoDB 'payment_mix' : {mongo_payment} Dokumente")
print(f"MongoDB 'demand_by_weather' : {mongo_weather} Dokumente")
```

```
MongoDB 'demand_weather_hourly': 744 Dokumente
MongoDB 'payment_mix' : 4 Dokumente
MongoDB 'demand_by_weather' : 3 Dokumente
```

```
In [12]: parquet_path, csv_path = storage.save_flatfiles(hourly_pd, "demand_weather_hourly")
storage.save_flatfiles(weather_pd, "demand_by_weather")

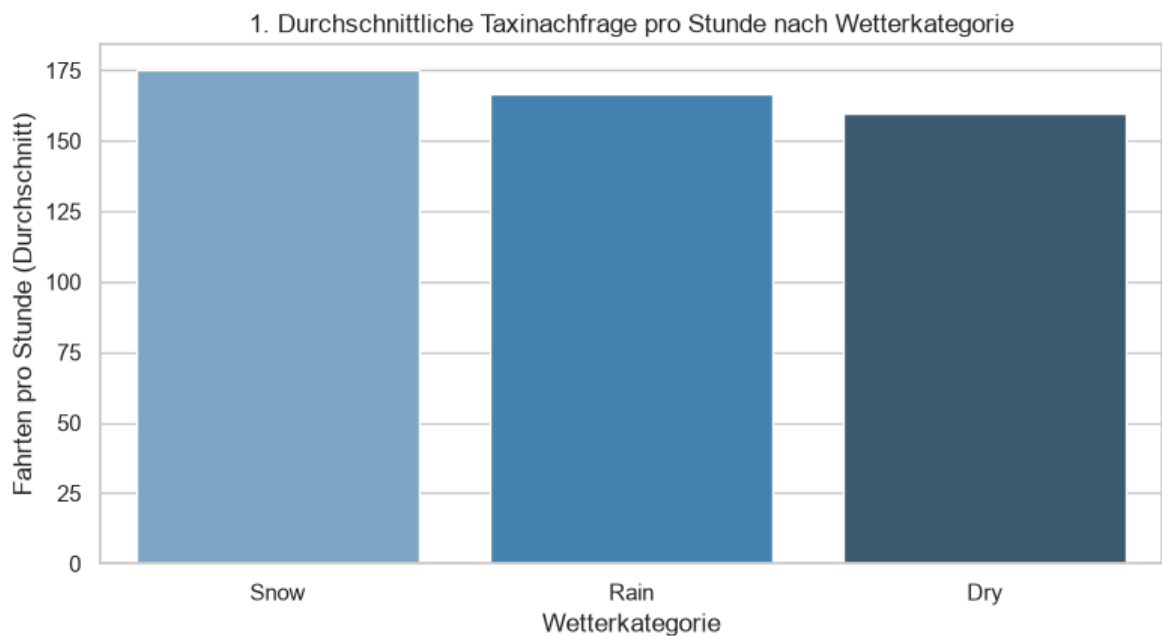
print("Flat Files gespeichert:")
print(" -", parquet_path.name)
print(" -", csv_path.name)
```

```
Flat Files gespeichert:
- demand_weather_hourly.parquet
- demand_weather_hourly.csv
```

9. Visualisierung & Data Story

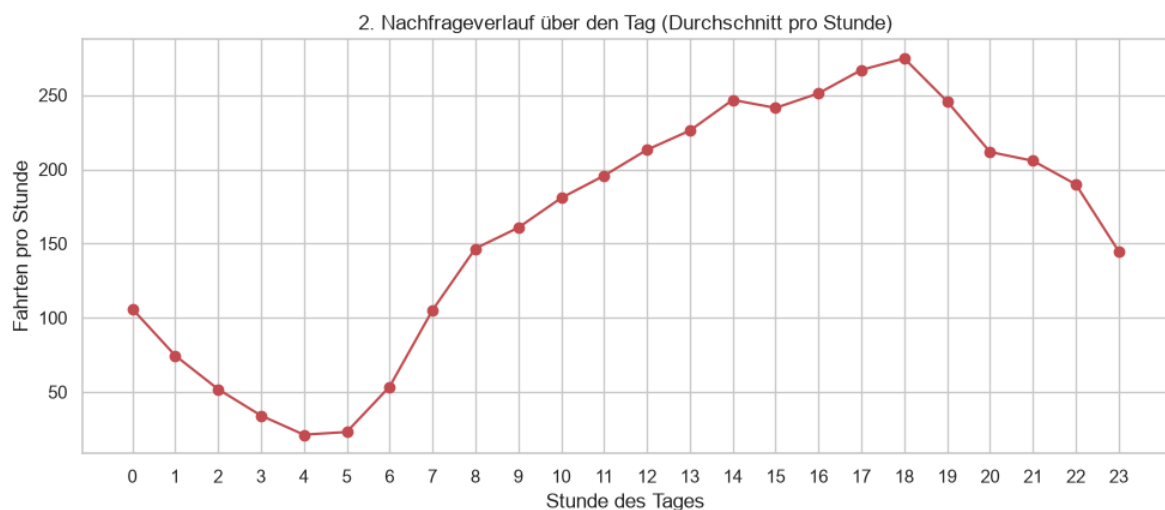
Wir erzählen die Geschichte hinter den Daten mit mehreren Visualisierungen, die alle auf den **von Spark verarbeiteten Ergebnissen** beruhen.

```
In [13]: fig, ax = plt.subplots(figsize=(8, 4.5))
order = weather_pd.sort_values("avg_trips_per_hour", ascending=False)
sns.barplot(data=order, x="weather_category", y="avg_trips_per_hour", hue="weath
ax.set_title("1. Durchschnittliche Taxinachfrage pro Stunde nach Wetterkategorie
ax.set_xlabel("Wetterkategorie")
ax.set_ylabel("Fahrten pro Stunde (Durchschnitt)")
plt.tight_layout()
plt.show()
```

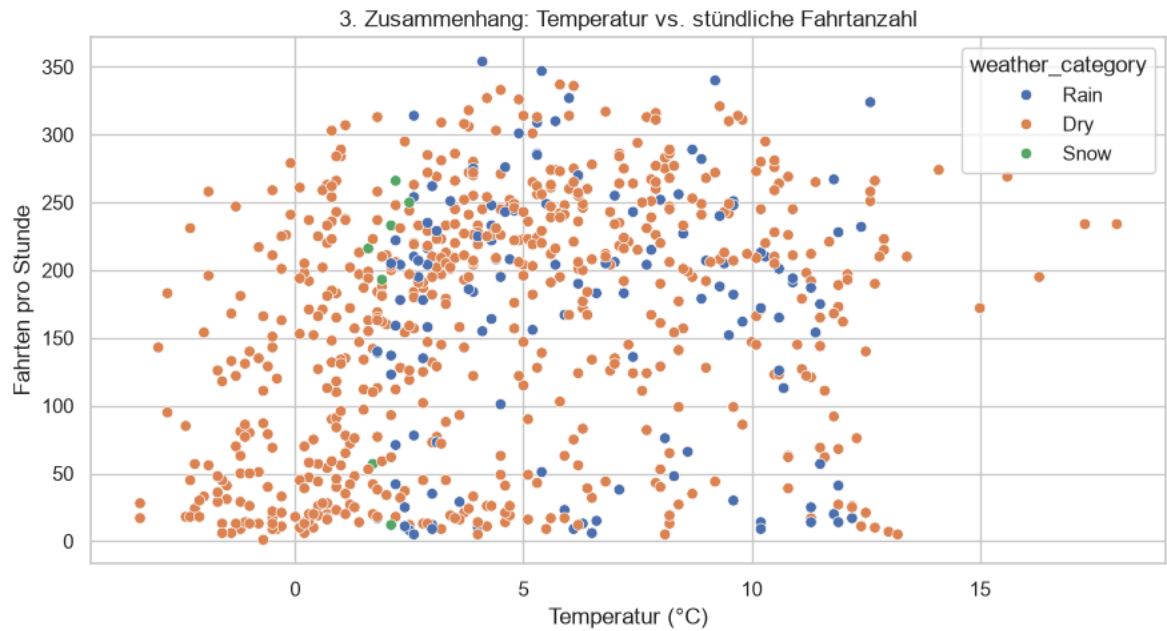


```
In [14]: hourly_profile = hourly_pd.groupby("hour")["trip_count"].mean()

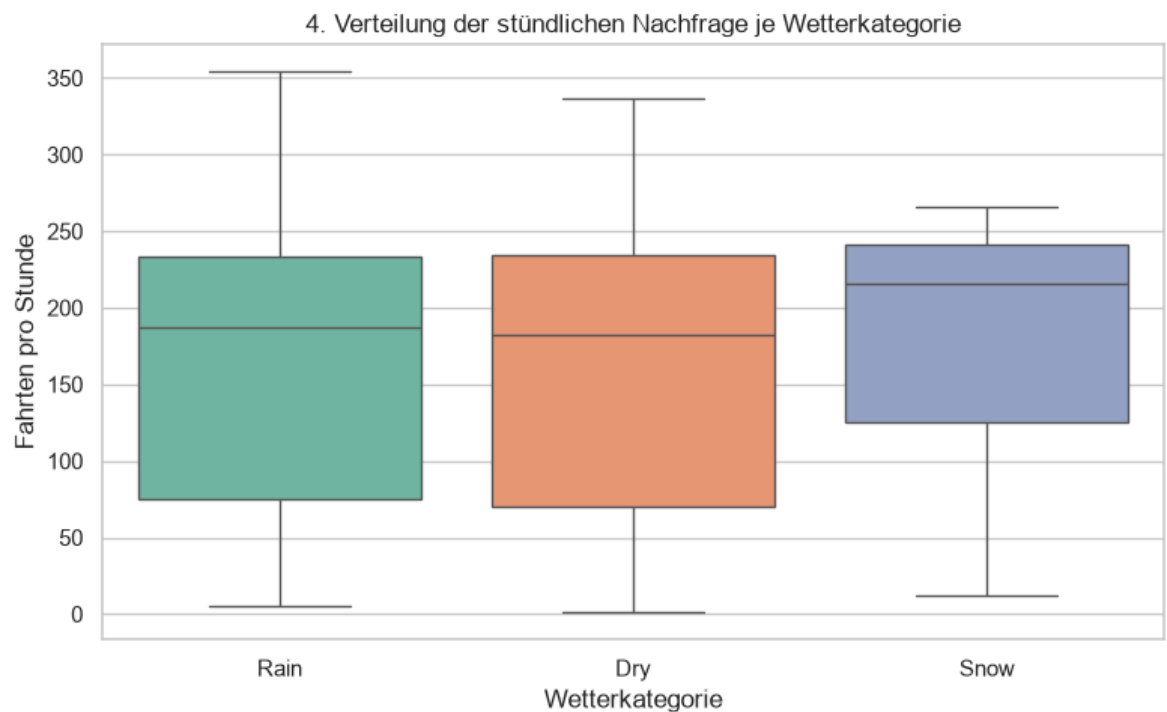
fig, ax = plt.subplots(figsize=(10, 4.5))
hourly_profile.plot(kind="line", marker="o", ax=ax, color="#C44E52")
ax.set_title("2. Nachfrageverlauf über den Tag (Durchschnitt pro Stunde)")
ax.set_xlabel("Stunde des Tages")
ax.set_ylabel("Fahrten pro Stunde")
ax.set_xticks(range(0, 24))
plt.tight_layout()
plt.show()
```



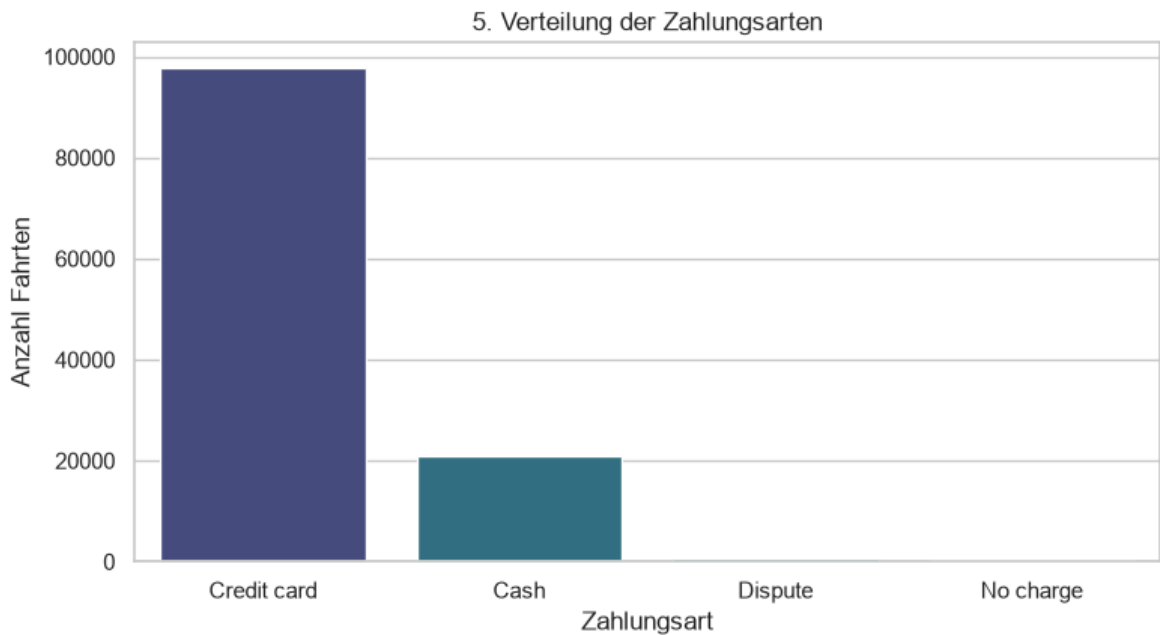

```
In [15]: fig, ax = plt.subplots(figsize=(9, 5))
sns.scatterplot(data=hourly_pd, x="temp_c", y="trip_count", hue="weather_category")
ax.set_title("3. Zusammenhang: Temperatur vs. stündliche Fahrtanzahl")
ax.set_xlabel("Temperatur (°C)")
ax.set_ylabel("Fahrten pro Stunde")
plt.tight_layout()
plt.show()
```



```
In [16]: fig, ax = plt.subplots(figsize=(8, 5))
sns.boxplot(data=hourly_pd, x="weather_category", y="trip_count", hue="weather_c")
ax.set_title("4. Verteilung der stündlichen Nachfrage je Wetterkategorie")
ax.set_xlabel("Wetterkategorie")
ax.set_ylabel("Fahrten pro Stunde")
plt.tight_layout()
plt.show()
```

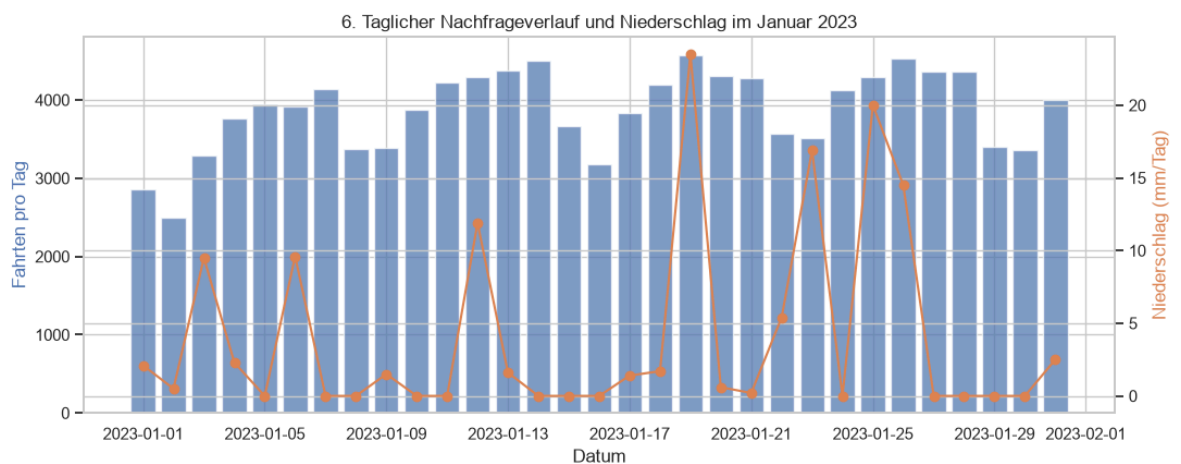


```
In [17]: fig, ax = plt.subplots(figsize=(8, 4.5))
sns.barplot(data=payment_pd, x="payment_label", y="trip_count", hue="payment_label")
ax.set_title("5. Verteilung der Zahlungsarten")
ax.set_xlabel("Zahlungsart")
ax.set_ylabel("Anzahl Fahrten")
plt.tight_layout()
plt.show()
```



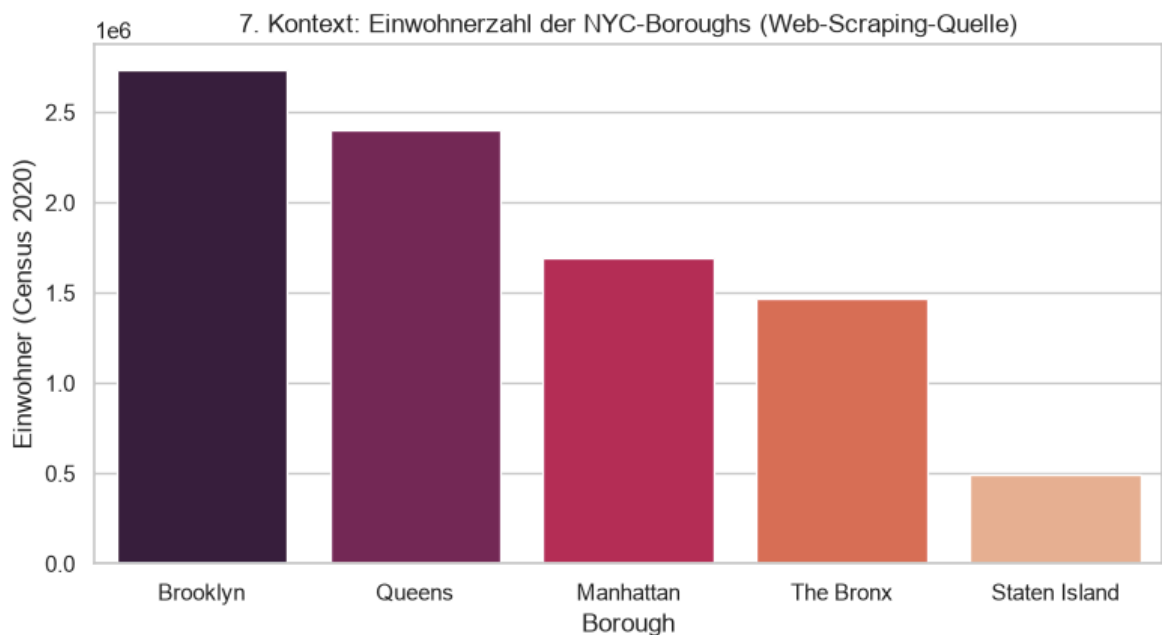
```
In [18]: daily = hourly_pd.set_index("date_hour").resample("D").agg(
trips=("trip_count", "sum"), precip=("precip_mm", "sum")
)

fig, ax1 = plt.subplots(figsize=(11, 4.5))
ax1.bar(daily.index, daily["trips"], color="#4C72B0", alpha=0.7, label="Fahrten")
ax1.set_ylabel("Fahrten pro Tag", color="#4C72B0")
ax1.set_xlabel("Datum")
ax2 = ax1.twinx()
ax2.plot(daily.index, daily["precip"], color="#DD8452", marker="o", label="Niederschlag")
ax2.set_ylabel("Niederschlag (mm/Tag)", color="#DD8452")
ax1.set_title("6. Taglicher Nachfrageverlauf und Niederschlag im Januar 2023")
plt.tight_layout()
plt.show()
```



```
In [19]: fig, ax = plt.subplots(figsize=(8, 4.5))
order_b = boroughs_df.sort_values("population_2020", ascending=False)
```

```
sns.barplot(data=order_b, x="borough", y="population_2020", hue="borough", legend=
ax.set_title("7. Kontext: Einwohnerzahl der NYC-Boroughs (Web-Scraping-Quelle)")
ax.set_xlabel("Borough")
ax.set_ylabel("Einwohner (Census 2020)")
plt.tight_layout()
plt.show()
```



9.1 Interpretation der Visualisierungen

1. **Nachfrage nach Wetterkategorie:** Anders als oft vermutet bricht die Taxinachfrage bei schlechtem Wetter nicht ein. Im Gegenteil – die durchschnittliche Anzahl der Fahrten pro Stunde liegt bei **Regen und Schnee tendenziell sogar leicht höher** als bei trockenem Wetter. Das deckt sich mit der Intuition, dass Menschen bei Niederschlag eher ein Taxi nehmen, statt zu Fuß zu gehen oder zu warten.
2. **Tagesverlauf:** Die Kurve zeigt die typischen Pendler-Spitzen am Morgen und am späten Nachmittag/Abend. Die Nachfrage ist stark zeitabhängig – ein zentrales Signal für die Flottensteuerung.
3. **Temperatur vs. Nachfrage:** Die Temperatur allein erklärt die Nachfrage kaum; die Punktwolke ist breit gestreut. Die Art des Wetters und die Tageszeit sind wichtigere Treiber als die reine Temperatur.
4. **Verteilung je Wetterkategorie:** Die Boxplots zeigen vergleichbare Streuungen über alle Kategorien, was bestätigt, dass die Nachfrage gegenüber dem Wetter erstaunlich robust ist.
5. **Zahlungsarten:** Kreditkarte dominiert deutlich vor Bargeld – relevant für die Abrechnungssysteme.
6. **Tagesverlauf & Niederschlag:** Auch im Monatsverlauf folgt die Nachfrage nicht sichtbar dem Niederschlag nach unten – die urbane Taxinachfrage in NYC ist wetterresistent.

7. **Borough-Kontext:** Brooklyn und Queens sind die bevölkerungsreichsten Bezirke und liefern den Rahmen für das enorme Mobilitätsaufkommen der Stadt.

10. Fazit und datengetriebene Erkenntnisse

Dieses Projekt hat eine vollständige **Big-Data-Engineering-Pipeline** umgesetzt: drei heterogene Datenquellen (Datei, REST-API, Web Scraping) wurden über **Apache Kafka** als zentralen Broker bereitgestellt, mit **Apache Spark** aus den Topics gelesen, geparkt, aggregiert und gejoint und schließlich in **MongoDB** sowie als **Flat Files** für die spätere Nutzung persistiert.

Wichtigste Erkenntnisse aus den Daten:

- **Wetterresistenz:** Die Taxinachfrage in NYC ist gegenüber dem Wetter bemerkenswert robust. Bei Regen und Schnee liegt die stündliche Nachfrage sogar leicht über dem Niveau trockener Stunden – Niederschlag verlagert die Mobilität offenbar eher ins Taxi.
- **Zeitliche Abhängigkeit:** Der dominierende Treiber der Nachfrage ist der **Pendler-Rhythmus** mit klaren Spitzen am Morgen und Abend, nicht das Wetter.
- **Temperatur als schwacher Prädiktor:** Die reine Temperatur erklärt die Nachfrage kaum; die Kombination aus Tageszeit und Wetterereignis ist aussagekräftiger.

Abschließendes Urteil: Aus Sicht des Data Engineering zeigt das Projekt, wie sich auch ein überschaubares Datenvolumen mit den Verfahren der Big-Data-Welt (Streaming-Broker, verteilte Verarbeitung, NoSQL-Persistenz) sauber strukturieren lässt. Ein Unternehmen, das Nachfrage- und Wetterdaten in Echtzeit über eine solche Pipeline verknüpft, könnte seine Flottenverfügbarkeit datengetrieben und dynamisch steuern.

```
In [20]: spark.stop()
         print("Spark-Session beendet. Pipeline erfolgreich abgeschlossen.")
```

Spark-Session beendet. Pipeline erfolgreich abgeschlossen.